Trabalho Computacional 2 – MS211

Amanda Rehder Cedro – RA: 253507

2025-11-27

Table of contents

1 INTRODUÇÃO **3**

2 QUESTÃO 1 - Interpolação Polinomial (Lagrange e Spline Linear) **4**
2.1 Explicação do conteúdo: . . . 4
2.2 *Resolução da questão:* . . . 7
2.3 Questão 1a: . . . 7
2.4 Questão 1b: . . . 9

3 QUESTÃO 2 - Ajuste de Curvas pelo Método dos Quadrados Mínimos **13**
3.1 Explicação do conteúdo: . . . 13
3.2 *Resolução da questão:* . . . 15
3.3 Questão 2: . . . 15

4 QUESTÃO 3 - Integração Numérica (Trapézio e Simpson) **18**
4.1 Explicação do conteúdo: . . . 18
4.2 *Resolução da questão:* . . . 19
4.3 Questão 3a: . . . 19
4.4 Questão 3b: . . . 20

5 QUESTÃO 4 - EDOs – Problemas de Valor Inicial (Euler e Runge-Kutta) **22**
5.1 Explicação do conteúdo: . . . 22
5.2 *Resolução da questão:* . . . 23
5.3 *Questão 4a - MÉTODO DE EULER:* . . . 23
5.4 *Questão 4b - MÉTODO DE RUNGE-KUTTA DE 2ª ORDEM:* . . . 25
5.5 *Questão 4c:* . . . 27
5.6 *Questão 4d - ANÁLISE PARA INTERVALOS LONGOS:* . . . 29

6 QUESTÃO 5 - EDOs – Problemas de Valor de Contorno (Diferenças Finitas) **34**
6.1 Explicação do conteúdo: . . . 34

6.2 *Resolução da questão:* 37
6.3 Questão 5a: 37
6.4 Questão 5b: 41
6.5 Questão 5c: 41
6.6 Questão 5d: 45

7 CONCLUSÃO: 47

8 BIBLIOGRAFIA: 48

1 INTRODUÇÃO

O trabalho tem como objetivo desenvolver competências em implementação numérica, compreender fundamentos teóricos dos métodos numéricos e desenvolver a capacidasde de análise crítica a partir da criação de códigos para as questões e a análise dos resultados. Para que isso fosse possível, foi utilizado o Software RStudio juntamente da base R para implementação de gráfico.

2 QUESTÃO 1 - Interpolação Polinomial (Lagrange e Spline Linear)

2.1 Explicação do conteúdo:

- Interpolação é encontrar uma função que dado um conjunto de pontos (Xn, Yn): passe exatamente por todos os pontos e estime valores entre pontos conhecidos.
- Algumas de suas aplicações são: Prever valores em dados experimentais, Transformar digital para analógico, analisar de dados meteorológicos e computação gráfica (curvas suaves).

Interpolação Polinomial de Lagrange:

- A ideia é encontrar um polinômio único que passe por todos os n pontos.
- Formula:

Para n pontos (X0, Y0), (X1,Y1), ..., (Xn,Yn) :

$$P(x) = \sum_{i=0}^{n} y_i \cdot L_i(x)$$

Os polinômios de Lagrange Li(X) são:

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

Exemplo com 3 pontos:

Para (X0, Y0), (X1,Y1), (X2,Y2):

$$\begin{aligned} P(x) = y_0 \cdot \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} \\ + y_1 \cdot \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} \\ + y_2 \cdot \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} \end{aligned}$$

A interpolação Polinomial de Lagrange possui vantagens e desvantagens:

- **Vantagens:** A fórmula explícita e elegante, é facil de implementar, o polinômio é único e suave
- **Desvantagens:** O fenômeno de Runge gera oscilações fortes em extremos, é computacionalmente caro para muitos pontos, para aicionar um ponto requer recalcular tudo.

- **ERRO:** O erro depende da (n+1)-ésima derivada da função e pode ter oscilações grandes entre pontos

EXEMPLO NO R:

```
# Interpolação de Lagrange
lagrange <- function(x_points, y_points, x) {
  n <- length(x_points)
  result <- 0
  for (i in 1:n) {
    # Calcular L_i(x)
    L <- 1
    for (j in 1:n) {
      if (j != i) {
        L <- L * (x - x_points[j]) / (x_points[i] - x_points[j])
      }
    }
    result <- result + y_points[i] * L
  }
  return(result)
}
# Exemplo de uso
x_points <- c(1, 2, 4)
y_points <- c(1, 4, 16)
# Calcular em x = 3
valor <- lagrange(x_points, y_points, 3)
print(paste("P(3) =", valor))  # Deve ser próximo de 9
```

```
[1] "P(3) = 9"
```

Spline Linear:

- A ideia é conectar os pontos com segmentos de reta em vez de um polinômio único. É um método mais robusto.
- Formula:

$$S_i(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i} \cdot (x - x_i), \quad \text{para } x \in [x_i, x_{i+1}]$$

A interpolação Polinomial de Lagrange possui vantagens e desvantagens:

- **Vantagens:** É omputacionalmente eficiente, é etável (sem oscilações), é facil de atualizar (adicionar pontos)
- **Desvantagens:** Não é suave (as mudanças bruscas na derivada), a aroximação menos precisa
- **ERRO:** O erro é proporcional ao quadrado do espaçamento entre ponto, é mais previsível e controlável

EXEMPLO NO R:

```r
# Spline Linear
spline_linear <- function(x_points, y_points, x) {
  n <- length(x_points)

  # Encontrar em qual intervalo x está
  for (i in 1:(n-1)) {
    if (x >= x_points[i] & x <= x_points[i+1]) {
      # Calcular spline linear neste intervalo
      m <- (y_points[i+1] - y_points[i]) / (x_points[i+1] - x_points[i])
      return(y_points[i] + m * (x - x_points[i]))
    }
  }

  stop("x fora do intervalo")
}

# Versão que retorna a função completa
spline_linear_completo <- function(x_points, y_points) {
  n <- length(x_points)

  function(x) {
    for (i in 1:(n-1)) {
      if (x >= x_points[i] & x <= x_points[i+1]) {
        m <- (y_points[i+1] - y_points[i]) / (x_points[i+1] - x_points[i])
        return(y_points[i] + m * (x - x_points[i]))
      }
    }
    return(NA)
  }
}

# Exemplo de uso
x_points <- c(1, 2, 4, 5)
```

```r
y_points <- c(1, 4, 16, 25)

spline_func <- spline_linear_completo(x_points, y_points)
valor <- spline_func(3)
print(paste("S(3) =", valor))
```

```
[1] "S(3) = 10"
```

DICAS:

- Usar Lagrange quando: Se tem poucos pontos (até 5-6), precisa-se de uma função suave, quando a precisão analítica é importante
- Usar Spline Linear quando: Se tem muitos pontos, quando a estabilidade é mais importante que suavidade, quando se tem preocupação com eficiência computacional

2.2 *Resolução da questão:*

2.3 Questão 1a:

Dados os pontos (x0, f(x0)), (x1, f(x1)), ..., (xn, f(xn)), portanto (n+1) pontos, deve-se aproximar um polinômio pn(x), de grau menor ou igual a n, a uma função f(x), tal que f(xk) = pn(xk), com k = 0, 1, 2, ..., n. Existe sempre um polinômio pn(x) que satisfaça estas condições? Caso exista, ele é único? Explique.

Para que satisfaça as condições, precisamos que o polinômigo garanta, existência e unicidade.

- Existência: Teorema fundamental da interpolação polinomial: Dados (n+1) popntos distintos (x0, y0), (x1, y1), ..., (xn, yn), existe sempre um polinômio pn(x) de grau no máximo n que interpola esses pontos, sendo assim, existe

$$p_n(x_k) = y_k, \quad \text{para } k = 0, 1, \dots, n$$

- Unicidade: O polinômio interpolador pn(x) é único. Supondo a existência de dois polinômios distintos, ambos de grau menor ou iguasl a n, que interpolam os mesmos (x+1) pontos. A diferença entre esses dois polinômios seriam um polinômio de grau menor ou igual a n, com (n+1) raízes. Pelo Teorema fundamental da álgebra, um polinômio pode ter no máximo n raízes, o que gera uma contradição com o que foi proposto anteriormente. Assim, ambos os polinômios que interpolam os (x+1) são iguais. Esse polinômio interpolador pode ser expresso de diversas formas equivalentes (Lagrange, Newton e forma padrão)

EXEMPLO:

```
x_points <- c(1, 2, 4)
y_points <- c(1, 4, 16)
# Matriz de Vandermonde
V <- matrix(c(
  1, x_points[1], x_points[1]^2,
  1, x_points[2], x_points[2]^2,
  1, x_points[3], x_points[3]^2
), nrow = 3, byrow = TRUE)
# Cálculos
det_V <- det(V)
coeficientes <- solve(V, y_points)
# Função do polinômio
polinomio <- function(x) {
  coeficientes[1] + coeficientes[2]*x + coeficientes[3]*x^2
}
# Truque: Captura a matriz como string para não quebrar o bloco visual
matriz_str <- paste(capture.output(print(V)), collapse = "\n")
# Monta o texto de verificação (o loop)
verificacao_str <- ""
for (i in 1:3) {
  linha <- sprintf("p(%d) = %.6f, valor esperado = %d\n",
                   x_points[i], polinomio(x_points[i]), y_points[i])
  verificacao_str <- paste0(verificacao_str, linha)
}

relatorio <- paste0(
  "Exemplo com 3 pontos distintos:\n",
  "Pontos x: ", paste(x_points, collapse = ", "), "\n",
  "Pontos y: ", paste(y_points, collapse = ", "), "\n\n",

  "Matriz de Vandermonde V:\n",
  matriz_str, "\n\n", # Aqui entra a matriz convertida em texto

  "Determinante de V: ", det_V, "\n",
  "Como det(V) != 0, a matriz é invertível\n\n",

  "Coeficientes do polinômio interpolador:\n",
  "a0 = ", round(coeficientes[1], 6), "\n",
  "a1 = ", round(coeficientes[2], 6), "\n",
  "a2 = ", round(coeficientes[3], 6), "\n\n",
```

```
  "Verificação da interpolação:\n",
  verificacao_str
)
cat(relatorio)
```

```
Exemplo com 3 pontos distintos:
Pontos x: 1, 2, 4
Pontos y: 1, 4, 16

Matriz de Vandermonde V:
     [,1] [,2] [,3]
[1,]    1    1    1
[2,]    1    2    4
[3,]    1    4   16

Determinante de V: 6
Como det(V) != 0, a matriz é invertível

Coeficientes do polinômio interpolador:
a0 = 0
a1 = 0
a2 = 1

Verificação da interpolação:
p(1) = 1.000000, valor esperado = 1
p(2) = 4.000000, valor esperado = 4
p(4) = 16.000000, valor esperado = 16
```

Os pontos satisfazem os critérios de existênica - já que para quaisquer (n+1) pontos distintos, existe um polinômio interpolador de grau n - e unicidade, já que O polinômio interpolador é único, demonstrado pela invertibilidade da matriz de Vandermonde ou pelo argumento das raízes múltiplas. Para que isso ocorra, os pontos x0, x1, ..., xn devem ser distintos. Se houver pontos repetidos, a matriz de Vandermonde será singular e o polinômio interpolador pode não existir ou não ser único.

2.4 Questão 1b:

```
# --- 1. Definições e Funções ---
x <- c(0.4, 0.52, 0.6)
f <- c(0.27, 0.29, 0.32)
```

```r
x0 <- 0.47
# Função de Lagrange
lagrange_eval <- function(x, f, x0) {
  n <- length(x)
  p <- 0
  for (k in 1:n) {
    Lk <- 1
    for (j in 1:n) {
      if (j != k) {
        Lk <- Lk * (x0 - x[j]) / (x[k] - x[j])
      }
    }
    p <- p + f[k] * Lk
  }
  return(p)
}
# --- 2. Cálculos---
p_x0 <- lagrange_eval(x, f, x0)
# Estimativa do erro (Cálculo das Diferenças Divididas)
x_all <- c(0.34, 0.4, 0.52, 0.6)
f_all <- c(0.22, 0.27, 0.29, 0.32)
f1 <- diff(f_all) / diff(x_all)
f2 <- diff(f1) / (x_all[3:4] - x_all[1:2])
f3 <- diff(f2) / (x_all[4] - x_all[1])
D3 <- f3
erro_est <- D3 * (x0 - x[1]) * (x0 - x[2]) * (x0 - x[3])
val_real_est <- p_x0 + erro_est
tabela_pontos <- paste(capture.output(print(data.frame(x, f))), collapse = "\n")
termos_poly <- c()
for (k in 1:length(x)) {
  termos_str <- paste0("(x - ", x[-k], ")", collapse = "")
  denom_str <- paste0("(", paste(x[k] - x[-k], collapse = "*"), ")")
  bloco <- sprintf("%.5f*%s / %s", f[k], termos_str, denom_str)
  termos_poly <- c(termos_poly, bloco)
}
polinomio_completo <- paste(termos_poly, collapse = " + ")
relatorio <- paste0(
  "Pontos utilizados:\n",
  tabela_pontos, "\n\n",
  sprintf("Valor interpolado p(%.2f) = %.10f\n\n", x0, p_x0),
  "Polinômio de Lagrange:\n",
  "p(x)=", polinomio_completo, "\n\n",
```

```
  sprintf("Estimativa do erro R2(%.2f) = %.10f\n", x0, erro_est),
  sprintf("Valor real estimado f(%.2f) ~ p(%.2f) + R2 = %.10f\n", x0, x0, val_real_est)
)
cat(relatorio)
```

```
Pontos utilizados:
     x    f
1 0.40 0.27
2 0.52 0.29
3 0.60 0.32

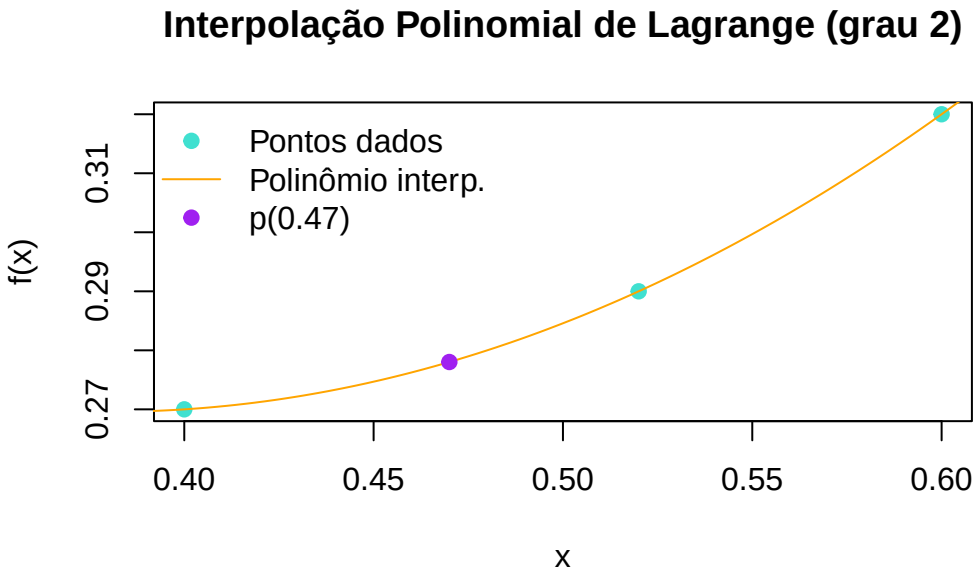
Valor interpolado p(0.47) = 0.2780208333

Polinômio de Lagrange:
p(x)=0.27000*(x - 0.52)(x - 0.6) / (-0.12*-0.2) + 0.29000*(x - 0.4)(x - 0.6) / (0.12*-
0.08) + 0.32000*(x - 0.4)(x - 0.52) / (0.2*0.08)

Estimativa do erro R2(0.47) = 0.0083043981
Valor real estimado f(0.47) ~ p(0.47) + R2 = 0.2863252315
```

```
# --- 5. Gráfico ---
xgrid <- seq(0.34, 0.72, length.out = 300)
ygrid <- sapply(xgrid, function(xx) lagrange_eval(x, f, xx))

plot(x, f, pch = 19, col = "turquoise",
     main = "Interpolação Polinomial de Lagrange (grau 2)",
     xlab = "x", ylab = "f(x)")
lines(xgrid, ygrid, col = "orange", lwd = 1)
points(x0, p_x0, col = "purple", pch = 19)
legend("topleft", legend = c("Pontos dados", "Polinômio interp.", "p(0.47)"),
       col = c("turquoise", "orange", "purple"), pch = c(19, NA, 19),
       lty = c(NA, 1, NA), bty = "n")
```


Interpolação Polinomial de Lagrange (grau 2)
Pontos dados
Polinômio interp.
p(0.47)
f(x)
0.27
0.29
0.31
0.40
0.45
0.50
0.55
0.60
x

3 QUESTÃO 2 - Ajuste de Curvas pelo Método dos Quadrados Mínimos

3.1 Explicação do conteúdo:

O método dos quadrados mínimos é uma técnica matemática usada para encontrar a melhor curva que aproxima um conjunto de dados experimentais. Procuramos a curva que melhor se aproxima dos pontos, medindo os valores de x e y com seus erros. Esse método minimiza a soma dos quadrados das diferenças entre valores observados e previstos, gera um sistema de equações lineares (sistema normal). Funciona para:

- retas
- polinômios
- funções linearizáveis
- modelos não lineares complexos

Método explicado computacionalmente:

```
# --- 1. Dados e Cálculos ---
# Dados (poderiam vir de um experimento)
x <- c(1, 2, 3, 4, 5)
y <- c(2, 4, 5, 4, 5)
n <- length(x)
# Somatórios básicos
Sx  <- sum(x)
Sy  <- sum(y)
Sxx <- sum(x^2)
Sxy <- sum(x * y)
# Fórmulas dos MQM
# Coeficiente angular 'a'
a <- (n * Sxy - Sx * Sy) / (n * Sxx - Sx^2)
# Coeficiente linear 'b'
b <- (Sy - a * Sx) / n
# --- 2. Preparação do Texto ---
relatorio <- paste0(
  "### AJUSTE DE RETA PELO MÉTODO DOS QUADRADOS MÍNIMOS ###\n\n",
  "1. Legenda dos Somatórios:\n",
  "Sx  = somatório de x\n",
  "Sy  = somatório de y\n",
  "Sxx = somatório de x^2\n",
  "Sxy = somatório de xy\n\n",
  "2. Resultados Calculados:\n",
```

```
  # Usando sprintf ou round para formatar os números e não ficarem gigantes
  sprintf("Coeficiente angular a = (n * Sxy - Sx * Sy) / (n * Sxx - Sx^2) = %.5f\n", a),
  sprintf("Coeficiente linear  b = (Sy - a * Sx) / n = %.5f\n\n", b),
  "3. Equação Final:\n",
  sprintf("y = %.5f * x + %.5f\n", a, b)
)
# --- 3. Impressão do Relatório ---
cat(relatorio)
```

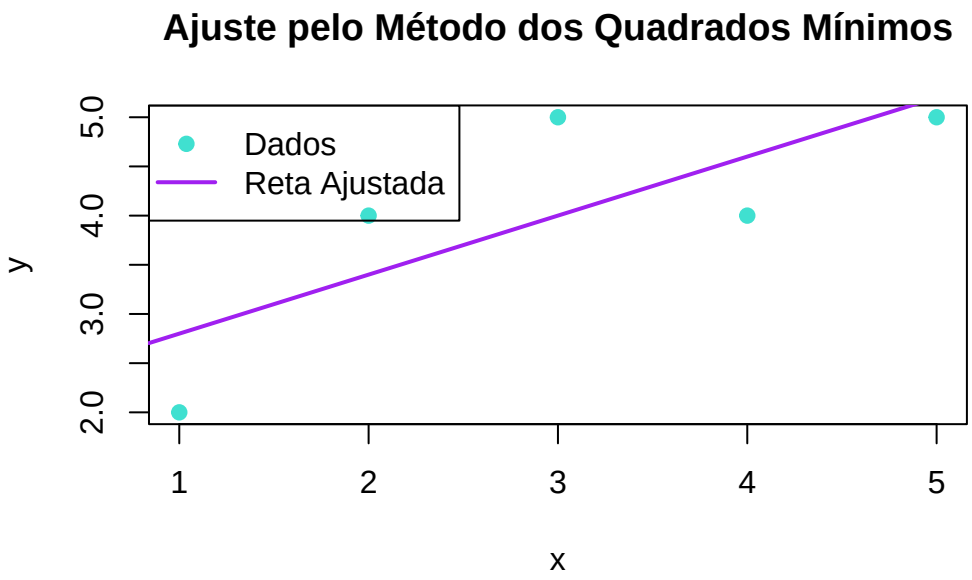
```
### AJUSTE DE RETA PELO MÉTODO DOS QUADRADOS MÍNIMOS ###

1. Legenda dos Somatórios:
Sx  = somatório de x
Sy  = somatório de y
Sxx = somatório de x^2
Sxy = somatório de xy

2. Resultados Calculados:
Coeficiente angular a = (n * Sxy - Sx * Sy) / (n * Sxx - Sx^2) = 0.60000
Coeficiente linear  b = (Sy - a * Sx) / n = 2.20000

3. Equação Final:
y = 0.60000 * x + 2.20000
```

```
# --- 4. Plotagem ---
plot(x, y, pch=19, col="turquoise", main="Ajuste pelo Método dos Quadrados Mínimos",
     xlab="x", ylab="y")
abline(a = b, b = a, col="purple", lwd=2)
legend("topleft", legend=c("Dados", "Reta Ajustada"),
       col=c("turquoise", "purple"), pch=c(19, NA), lty=c(NA,1), lwd=c(1,2))
```



3.2 *Resolução da questão:*

3.3 Questão 2:

```r
# 1. Dados da Tabela
x <- 1:8
y <- c(0.5, 0.6, 0.9, 0.8, 1.2, 1.5, 1.7, 2.0)
n <- length(x)
dados <- data.frame(x, y)

# 2(a). Ajuste Linear (reta: y = a*x + b)
fit_linear <- lm(y ~ x)
a_lin <- coef(fit_linear)[2]
b_lin <- coef(fit_linear)[1]
yhat_lin <- fitted(fit_linear)
res_lin <- residuals(fit_linear)
SSE_lin <- sum(res_lin^2)

msg_linear <- paste0(
  "QUESTÃO 2a - Ajuste Linear (Reta)\n",
  sprintf("Equação da reta ajustada:\n y = %.5f*x + %.5f\n", a_lin, b_lin),
```

```
  sprintf("Soma dos Quadrados dos Desvios = %.6f\n\n", SSE_lin)
)
cat(msg_linear)
```

```
QUESTÃO 2a - Ajuste Linear (Reta)
Equação da reta ajustada:
 y = 0.21667*x + 0.17500
Soma dos Quadrados dos Desvios = 0.088333
```

```
# 2(b). Ajuste Quadrático (parábola: y = a*x^2 + b*x + c)
fit_quad <- lm(y ~ I(x^2) + x)
a_quad <- coef(fit_quad)[2]
b_quad <- coef(fit_quad)[3]
c_quad <- coef(fit_quad)[1]
yhat_quad <- fitted(fit_quad)
res_quad <- residuals(fit_quad)
SSE_quad <- sum(res_quad^2)

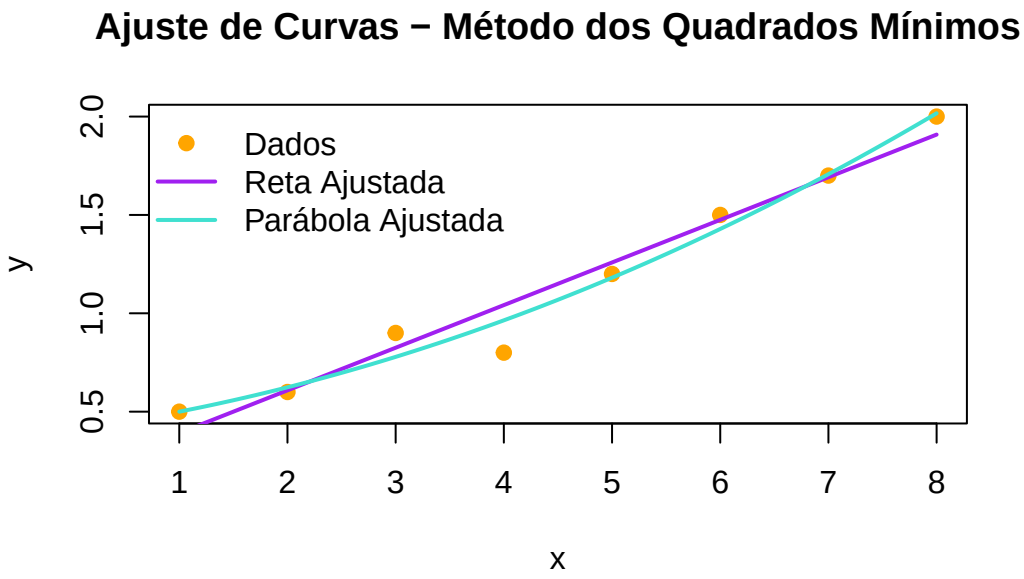
m2 <- paste0("QUESTÃO 2b - Ajuste Quadrático (Parábola)\n",
sprintf("Equação da parábola ajustada:\n y = %.5f*x^2 + %.5f*x + %.5f\n",
            a_quad, b_quad, c_quad),
sprintf("Soma dos Quadrados dos Desvios = %.6f\n\n", SSE_quad))
cat(m2)
```

```
QUESTÃO 2b - Ajuste Quadrático (Parábola)
Equação da parábola ajustada:
 y = 0.01548*x^2 + 0.07738*x + 0.40714
Soma dos Quadrados dos Desvios = 0.048095
```

```
# 3. Comparação entre os ajustes
m3 <- paste0(" QUESTÃO 2c - Comparação entre os Ajustes\n",
sprintf("Erro - Reta: %.6f\n", SSE_lin),
sprintf("Erro - Parábola: %.6f\n", SSE_quad),
sprintf("A parábola reduz o erro em %.2f%%\n\n",
            100 * (SSE_lin - SSE_quad) / SSE_lin))
cat(m3)
```

```
 QUESTÃO 2c - Comparação entre os Ajustes
Erro - Reta: 0.088333
Erro - Parábola: 0.048095
A parábola reduz o erro em 45.55%
```

```r
# 4. Gráfico de Dispersão + Reta + Parábola
xgrid <- seq(min(x), max(x), length.out = 300)
y_lin_grid <- a_lin * xgrid + b_lin
y_quad_grid <- a_quad * xgrid^2 + b_quad * xgrid + c_quad
plot(x, y, pch = 19, col = "orange",
     main = "Ajuste de Curvas - Método dos Quadrados Mínimos",
     xlab = "x", ylab = "y")
lines(xgrid, y_lin_grid, col = "purple", lwd = 2)
lines(xgrid, y_quad_grid, col = "turquoise", lwd = 2)
legend("topleft",
       legend = c("Dados", "Reta Ajustada", "Parábola Ajustada"),
       col = c("orange", "purple", "turquoise"),
       pch = c(19, NA, NA), lty = c(NA, 1, 1), lwd = c(NA, 2, 2),
       bty = "n")
```



4 QUESTÃO 3 - Integração Numérica (Trapézio e Simpson)

4.1 Explicação do conteúdo:

A integração numérica consiste em aproximar a área sob a curva usando figuras simples. Geralmente utilizado quando na integral, a f(x) é muito complexa, quando só temos pontos discretos e quando a primitiva não existe em forma elementar.

Regra do Trapézio:

- A ideia é aproximar a área sob a curva pela área de trapézios
- Usada quando a função é aproximadamente linear por partes, quando utilizamos dados experimentais com ruído e quando a implementação simples é prioritária
- Apresenta a forma simples: utilizad aquando se tem somente 1 intervalo,

$$\int_a^b f(x)\,dx \approx \frac{b-a}{2}\left[f(a)+f(b)\right]$$

  . Área do trapézio com bases f(a) e f(b) e altura (b—a)
- Trapézio Composta (n intervalos): Dividimos o intervalo $[a,b]$ em n subintervalos. Definimos o passo como $h=\frac{b-a}{n}$ e os pontos como $x_i = a + i \cdot h$, para $i = 0, 1, \ldots, n$.

A integral é aproximada pela fórmula:

$$\int_a^b f(x)\,dx \approx \frac{h}{2}\left[f(x_0)+2f(x_1)+2f(x_2)+\cdots+2f(x_{n-1})+f(x_n)\right]$$

- Erro: tem ordem quadrática, então diminui com o quadrado do tamanho do passo. Possui a seguinte fórmula:

$$E_t = -\frac{(b-a)^3}{12n^2}f''(\xi)$$

Regra de Simpson:

- A ideia é aproximar a função por arcos de parábola (polinômios de 2º grau) em vez de retas.
- Usada quando a função é suave (derivadas contínuas), quando a alta precisão é necessária e quando podemos garantir número par de subintervalos.
- **Simpson 1/3 Simples (2 subintervalos):** Precisamos de 3 pontos igualmente espaçados x_0, x_1, x_2. Sua fórmula é:

$$\int_{x_0}^{x_2} f(x)\,dx \approx \frac{h}{3}\left[f(x_0)+4f(x_1)+f(x_2)\right]$$

- **Simpson 1/3 Composta (n par):** Dividimos o intervalo em n subintervalos, onde n deve ser par. Sendo $h = \frac{b-a}{n}$ e $x_i = a + i \cdot h$, a integral é aproximada por:

$$\int_a^b f(x)\,dx \approx \frac{h}{3}\Big[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + ... \\ + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)\Big]$$

- Erro: tem ordem $O(h^4)$, por isso é muito mais preciso para funções suaves. Possui a seguinte fórmula:

$$E_s = -\frac{(b-a)^5}{180n^4} f^{(4)}(\xi)$$

DICAS:

- Simpson é geralmente mais preciso para o mesmo número de pontos
- Dobre o número de subintervalos para estimar o erro
- n deve ser par para Simpson 1/3
- Para funções periódicas, considere regra do trapézio com muitos pontos

4.2 *Resolução da questão:*

4.3 Questão 3a:

Em que sentido a regra de Simpson é melhor do que a regra dos Trapézios?

A regra de Simpson é melhor que a regra dos Trapézios principalmente porque tem ordem de precisão superior e erro assintoticamente muito menor para funções suaves. Para malhas com o mesmo passo **h** ou para o mesmo número de avaliações da função, Simpson normalmente produz aproximações muito mais precisas.

A regra dos Trapézios (elementar) integra exatamente polinômios até grau 1 (retas). A regra de Simpson (elementar) integra exatamente polinômios até grau 3 (cúbicos). Portanto, Simpson possui maior grau de exatidão. Simpson tem erro de ordem **(h^4)** enquanto Trapézios tem erro de ordem **(h^2)**, isto é, o erro de Simpson decresce muito mais rápido quando refinamos a malha.

Trapézio corresponde à integrar a interpolação linear em cada subintervalo. Simpson corresponde à integrar a interpolação quadrática (parábola) em pares de subintervalos. Interpoladores de grau mais alto capturam melhor a curvatura local da função, daí a maior precisão. Concluindo, a regra de Simpson é melhor no sentido de ter maior grau exato (exata até grau 3) e ordem de erro superior**(O(h^4))** vs **(O(h^2))** do trapézio.

Para funções suaves, isso significa muito menos erro para a mesma malha ou menos pontos necessários para atingir uma dada precisão.

4.4 Questão 3b:

```r
# Função integranda
f <- function(x) {
  3*(x^2) - 3*x + 1
}

# Valor exato da integral
I_exato <- 44

# 1. Regra dos Trapézios composta
trapz_rule <- function(f, a, b, n) {
  h <- (b - a) / n
  x <- seq(a, b, by = h)
  y <- f(x)
  I <- (h / 2) * (y[1] + 2 * sum(y[2:n]) + y[n + 1])
  return(I)
}

# 2. Regra de Simpson composta
simpson_rule <- function(f, a, b, n) {
  if (n %% 2 != 0) stop("O número de subintervalos (n) deve ser PAR para Simpson.")
  h <- (b - a) / n
  x <- seq(a, b, by = h)
  y <- f(x)
  I <- (h / 3) * (y[1] + 4 * sum(y[seq(2, n, 2)]) + 2 * sum(y[seq(3, n - 1, 2)]) + y[n + 1])
  return(I)
}

# 3. Aplicação: integral de 0 a 4 com n = 4 subintervalos
a <- 0; b <- 4; n <- 4
I_trap <- trapz_rule(f, a, b, n)
I_simp <- simpson_rule(f, a, b, n)
erro_trap <- abs(I_exato - I_trap)
erro_simp <- abs(I_exato - I_simp)

# 4. Saída formatada
relatorio <- paste0(
```

```
  "Integração Numérica - Regras dos Trapézios e Simpson\n",
  "Função: f(x) = 3x^2 - 3x + 1\n",
  "Intervalo: [0, 4]   Subintervalos: 4\n",
  "Valor exato: I = 44\n",
  sprintf("Regra dos Trapézios:  I_T = %.10f   Erro = %.10e\n", I_trap, erro_trap),
  sprintf("Regra de Simpson:     I_S = %.10f   Erro = %.10e\n\n", I_simp, erro_simp)
)

cat(relatorio)
```

```
Integração Numérica - Regras dos Trapézios e Simpson
Função: f(x) = 3x^2 - 3x + 1
Intervalo: [0, 4]   Subintervalos: 4
Valor exato: I = 44
Regra dos Trapézios:  I_T = 46.0000000000   Erro = 2.0000000000e+00
Regra de Simpson:     I_S = 44.0000000000   Erro = 0.0000000000e+00
```

```
# 5. Comparação (Agrupada)
if (erro_simp < erro_trap) {
  cat(sprintf("Comparação:\nA regra de Simpson é mais precisa (erro %.2e vs %.2e) \n",
              erro_simp, erro_trap))
} else {
  cat(sprintf("Comparação:\nA regra dos Trapézios é mais precisa (erro %.2e vs %.2e)\n",
              erro_trap, erro_simp))
}
```

```
Comparação:
A regra de Simpson é mais precisa (erro 0.00e+00 vs 2.00e+00)
```

5 QUESTÃO 4 - EDOs – Problemas de Valor Inicial (Euler e Runge-Kutta)

5.1 Explicação do conteúdo:

Precisamos de métodos numéricos para a resolução de problemas de valor incial pois muitas EDOS não tem solução analítica, essas soluções analíticas, quando existem, podem ser muito complexas e também, porque precisamos de respostas numéricas para aplicações práticas. Podemos resolver PVI's numéricamente com os métodos de Euler e Runge-Kutta.

- **Euler**: A ideia é proximar a curva solução por sua reta tangente em cada ponto.A fórmula é dada por:

$$y_{n+1} = y_n + h \cdot f(x_n, y_n)$$

h é o tamanho do passo e f(Xn,Yn) é a inclinação do ponto atual. Existem vantagens e desvantagens para esse método.

- Vantagens: É simples de implementar e fácil de entender.
- Desvantagens: É pouco preciso e o erro acumula rapidamente.

O erro no método de Euler é de primeira ordem.

- **Runge-Kutta**: A ideia é usar médias ponderadas de várias estimativas da inclinação para obter maior precisão. Existem duas formas.

Runge-Kutta de 2ª Ordem (RK2) – Método do Ponto Médio:

$$\begin{aligned} K_1 &= h \cdot f(x_n, y_n) \\ K_2 &= h \cdot f\left(x_n + \frac{h}{2}, y_n + \frac{K_1}{2}\right) \\ y_{n+1} &= y_n + K_2 \end{aligned}$$

O erro no método de RK2 é de segunda ordem, ou seja, $O(h^2)$.

Runge-Kutta de 4ª Ordem (RK4):

$$
\begin{aligned}
K_1 &= h \cdot f(x_n, y_n) \\
K_2 &= h \cdot f\left(x_n + \frac{h}{2}, y_n + \frac{K_1}{2}\right) \\
K_3 &= h \cdot f\left(x_n + \frac{h}{2}, y_n + \frac{K_2}{2}\right) \\
K_4 &= h \cdot f(x_n + h, y_n + K_3) \\
y_{n+1} &= y_n + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4)
\end{aligned}
$$

O erro no método de RK4 é de quarta ordem, ou seja, $O(h^4)$.

DICAS:

- Para a escolha do passo h: h muito grande = Pouca precisão, h muito pequeno = Muito tempo computacional. Uma solução é justar h durante a integração.
- Aplicações: Física- Movimento de partículas, Engenharia- Circuitos elétricos, Biologia- Modelos populacionais, Economia- Modelos de crescimento, Química- Cinética de reações.
- Sempre verifique a convergência diminuindo h
- RK4 é geralmente a melhor opção entre simplicidade e precisão
- Para sistemas de EDOs os métodos se estendem naturalmente
- Use métodos adaptativos para problemas com escalas de tempo diferentes

Os métodos de Euler e Runge-Kutta são ferramentas fundamentais para resolver EDOs numericamente. Enquanto Euler é ótimo para entender o conceito, RK4 é o método prático mais utilizado no mundo real por seu equilíbrio entre precisão e eficiência.

5.2 *Resolução da questão:*

5.3 *Questão 4a - MÉTODO DE EULER:*

```r
# Questão 4a
# Função Euler para y' = 0.04 y
euler_method <- function(h) {
  f <- function(x, y) 0.04 * y
  x <- seq(0, 1, by = h)
  n <- length(x)
  y <- numeric(n)
  y[1] <- 1000   # condição inicial

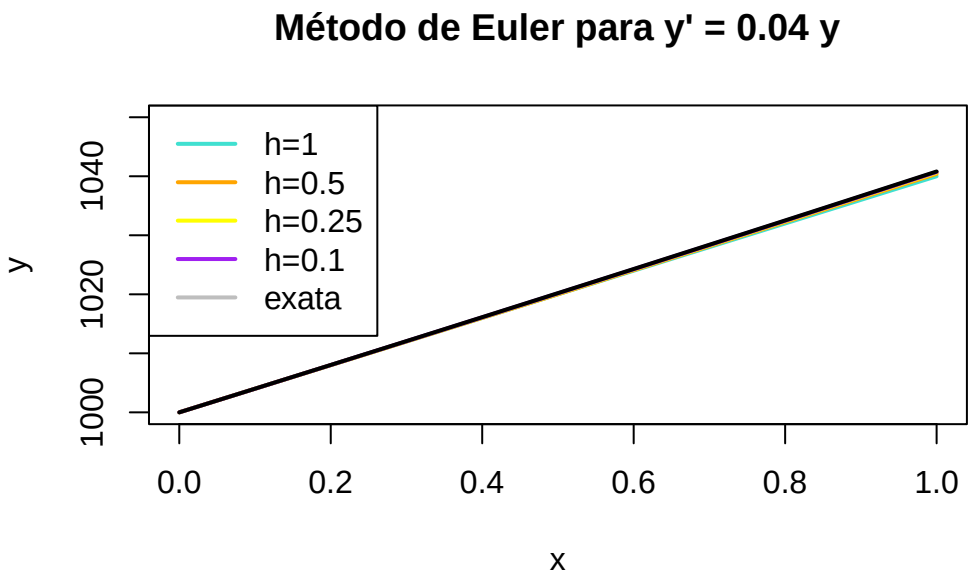
  for (k in 1:(n-1)) {
    y[k+1] <- y[k] + h * f(x[k], y[k])
```

```
  }
  return(data.frame(x = x, y = y))
}

# Soluções para cada h
h_values <- c(1, 0.5, 0.25, 0.1)
resultados <- lapply(h_values, euler_method)
names(resultados) <- paste0("h=", h_values)
# Solução exata em x=1
y_exato <- 1000 * exp(0.04)
# Tabela final dos valores aproximados
tabela <- data.frame(
  h = h_values,
  y_aprox = sapply(resultados, function(df) tail(df$y, 1)),
  y_exato = y_exato,
  erro = abs(sapply(resultados, function(df) tail(df$y, 1)) - y_exato)
)
print(tabela)
```

```
          h  y_aprox  y_exato       erro
h=1    1.00 1040.000 1040.811 0.81077419
h=0.5  0.50 1040.400 1040.811 0.41077419
h=0.25 0.25 1040.604 1040.811 0.20676419
h=0.1  0.10 1040.728 1040.811 0.08304017
```

```
# Gráfico
plot(NULL, xlim=c(0,1), ylim=c(1000,1050),
     xlab="x", ylab="y", main="Método de Euler para y' = 0.04 y")
colors <- c("turquoise", "orange", "yellow", "purple")
for (i in 1:length(h_values)) {
  lines(resultados[[i]]$x, resultados[[i]]$y, col=colors[i], lwd=2)
}
curve(1000*exp(0.04*x), add=TRUE, lwd=2)  # solução exata
legend("topleft", legend=c("h=1","h=0.5","h=0.25","h=0.1","exata"),
       col=c(colors,"gray"), lwd=2)
```



5.4 *Questão 4b - MÉTODO DE RUNGE-KUTTA DE 2ª ORDEM:*

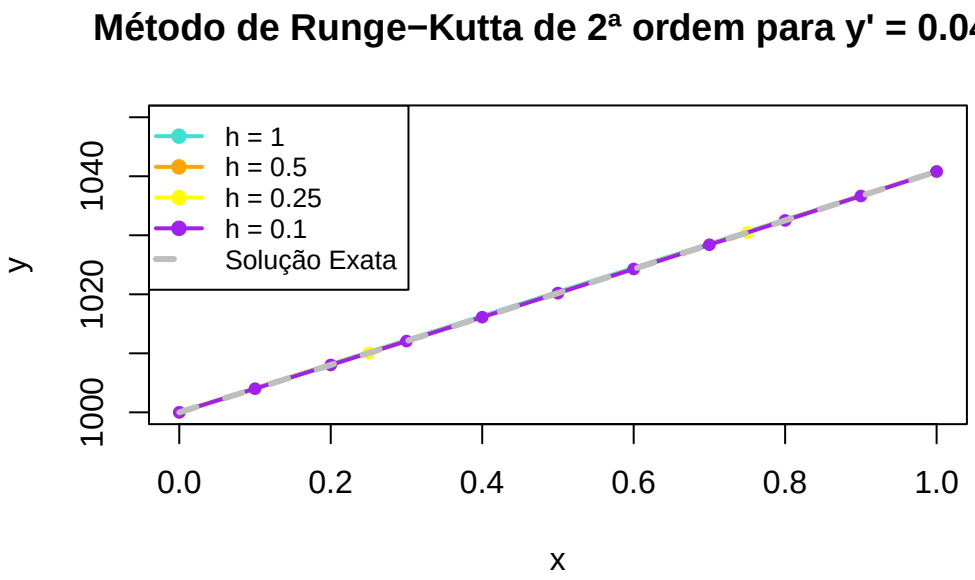
```r
# Questão - 4b
# Função Runge-Kutta de 2ª ordem (Método do Ponto Médio) para y' = 0.04 y
rk2_method <- function(h) {
  f <- function(x, y) 0.04 * y
  x <- seq(0, 1, by = h)
  n <- length(x)
  y <- numeric(n)
  y[1] <- 1000   # condição inicial

  for (k in 1:(n-1)) {
    k1 <- f(x[k], y[k])
    k2 <- f(x[k] + h/2, y[k] + (h/2) * k1)
    y[k+1] <- y[k] + h * k2
  }
  return(data.frame(x = x, y = y))
}
# Soluções para cada h
h_values <- c(1, 0.5, 0.25, 0.1)
resultados_rk2 <- lapply(h_values, rk2_method)
```

```r
names(resultados_rk2) <- paste0("h=", h_values)
# Solução exata em x=1
y_exato <- 1000 * exp(0.04)
# Tabela final dos valores aproximados
tabela_rk2 <- data.frame(
  h = h_values,
  y_aprox = sapply(resultados_rk2, function(df) tail(df$y, 1)),
  y_exato = y_exato,
  erro = abs(sapply(resultados_rk2, function(df) tail(df$y, 1)) - y_exato)
)
print(tabela_rk2)
```

```
          h  y_aprox  y_exato         erro
h=1    1.00 1040.800 1040.811 0.0107741924
h=0.5  0.50 1040.808 1040.811 0.0027341924
h=0.25 0.25 1040.810 1040.811 0.0006886904
h=0.1  0.10 1040.811 1040.811 0.0001106873
```

```r
# Gráfico
plot(NULL, xlim = c(0, 1), ylim = c(1000, 1050),
     xlab = "x", ylab = "y",
     main = "Método de Runge-Kutta de 2ª ordem para y' = 0.04 y")
colors <- c("turquoise", "orange", "yellow", "purple")
for (i in 1:length(h_values)) {
  lines(resultados_rk2[[i]]$x, resultados_rk2[[i]]$y, col = colors[i], lwd = 2,
        type = "o", pch = 19, cex = 0.6)
}
curve(1000 * exp(0.04 * x), add = TRUE,
      lwd = 3, col = "gray", lty = 2)# solução exata
legend("topleft",
       legend = c("h = 1", "h = 0.5", "h = 0.25", "h = 0.1", "Solução Exata"),
       col = c(colors, "gray"),
       lwd = c(rep(2, 4), 3),
       lty = c(rep(1, 4), 2),
       pch = c(rep(19, 4), NA),
       cex = 0.8)
```



5.5 *Questão 4c:*

```
library(ggplot2)
```

```
Warning: pacote 'ggplot2' foi compilado no R versão 4.5.2
```

```
# solução exata
xgrid <- seq(0,1,length.out=200)
y_exata <- 1000*exp(0.04*xgrid)

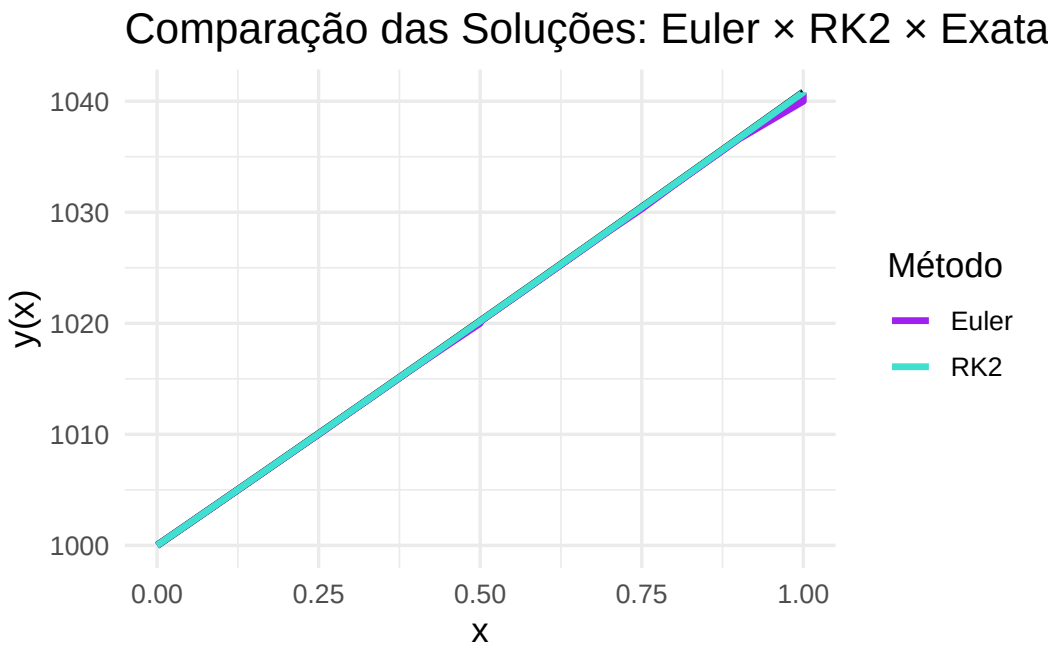
# pegar resultados existentes
df_euler  <- do.call(rbind, resultados)
df_euler$method <- "Euler"

df_rk2    <- do.call(rbind, resultados_rk2)
df_rk2$method <- "RK2"

df_plot <- rbind(df_euler, df_rk2)

ggplot() +
```

```
  geom_line(aes(x=xgrid, y=y_exata), color="black", linewidth=1.3) +
  geom_line(data=df_plot, aes(x=x, y=y, color=method), linewidth=1.2) +
  scale_color_manual(values=c("Euler"="purple","RK2"="turquoise")) +
  labs(title="Comparação das Soluções: Euler × RK2 × Exata",
       x="x", y="y(x)", color="Método") +
  theme_minimal(base_size = 13)
```




```
y_exato <- 1000 * exp(0.04)

tabela4c <- data.frame(
  h = h_values,
  Euler = sapply(resultados, function(df) tail(df$y, 1)),
  Erro_Euler = abs(sapply(resultados, function(df) tail(df$y, 1)) - y_exato),
  RK2 = sapply(resultados_rk2, function(df) tail(df$y, 1)),
  Erro_RK2 = abs(sapply(resultados_rk2, function(df) tail(df$y, 1)) - y_exato)
)

print(tabela4c, digits=10)
```

```
          h       Euler    Erro_Euler          RK2        Erro_RK2
h=1    1.00 1040.000000 0.81077419239 1040.800000 0.0107741923882
h=0.5  0.50 1040.400000 0.41077419239 1040.808040 0.0027341923881
```

```
h=0.25 0.25 1040.604010 0.20676419239 1040.810086 0.0006886903832
h=0.1  0.10 1040.727734 0.08304017348 1040.810664 0.0001106872833
```

Comparação com a solução exata:

- A solução exata é suave (exponencial) e as hipóteses de consistência das fórmulas numéricas estão satisfeitas.
- O método de Euler é de ordem 1: o erro global é proporcional a h, para se obter uma boa precisão é necessário um h pequeno. - O RK2 é de ordem 2: o erro global é proporcional a h^2. Assim, reduzindo h pela metade, o erro diminui aproximadamente por fator 4. Isso explica por que RK2, com o mesmo h, é muito mais preciso que Euler.
- Para essa EDO que é simples ambos métodos são estáveis com os passos usados; não houve crescimento numérico indesejado.
- Em termos práticos, RK2 dá resultados muito próximos da solução exata já com passos moderados; Euler exige passos bem menores para obter precisão comparável.
- RK2 tem 7um custo mais alto, pois faz duas avaliações por passo, no entanto, permite o uso de um h muito maior e ainda assim, obter um erro menor. Portanto, RK2 é quase sempre mais eficiente para alcançar precisões fixas.
- Euler é aceitável para entendimento e quando a precisão exigida é baixa, ou para prototipagem rápida; mas para resultados confiáveis é preferível o uso métodos de ordem maior.

5.6 *Questão 4d - ANÁLISE PARA INTERVALOS LONGOS:*

```r
# Questão 4d - Análise para intervalos longos

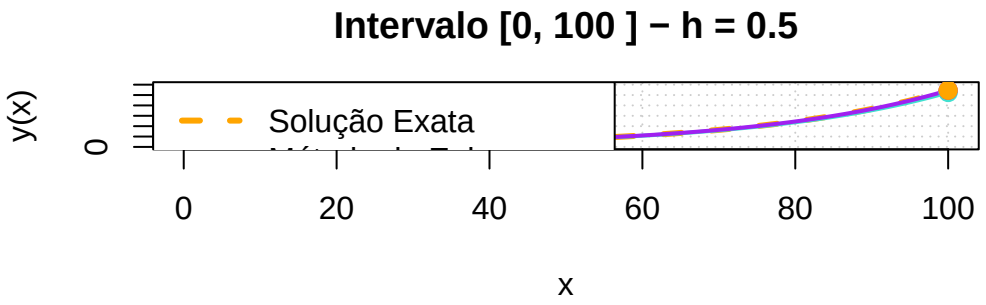
euler_method <- function(h, x_final = 1) {
  f <- function(x, y) 0.04 * y
  x <- seq(0, x_final, by = h)
  n <- length(x)
  y <- numeric(n)
  y[1] <- 1000
  for (k in 1:(n-1)) {
    y[k+1] <- y[k] + h * f(x[k], y[k])
  }
  return(data.frame(x = x, y = y))
}
rk2_method <- function(h, x_final = 1) {
  f <- function(x, y) 0.04 * y
  x <- seq(0, x_final, by = h)
  n <- length(x)
```

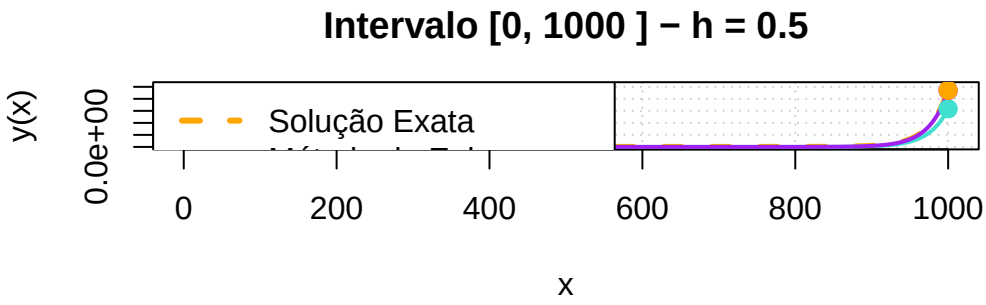
```r
  y <- numeric(n)
  y[1] <- 1000
  for (k in 1:(n-1)) {
    k1 <- f(x[k], y[k])
    k2 <- f(x[k] + h/2, y[k] + (h/2) * k1)
    y[k+1] <- y[k] + h * k2
  }
  return(data.frame(x = x, y = y))
}
# Parâmetros
h_values <- c(1, 0.5, 0.25, 0.1)
intervalos <- c(100, 1000)
# Análise para cada intervalo
for (x_final in intervalos) {
  cat(" INTERVALO 0 < x  ", x_final, "\n")
  y_exato <- 1000 * exp(0.04 * x_final)
  tabela <- data.frame(
    h = h_values,
    Euler = sapply(h_values, function(h) round(tail(euler_method(h, x_final)$y,1), 2)),
    Runge_Kutta = sapply(h_values, function(h) round(tail(rk2_method(h, x_final)$y, 1), 2)),
    Exato = round(y_exato, 2)
  )
  print(tabela)
  cat("\n")
}
```

```
 INTERVALO 0 < x   100
     h    Euler Runge_Kutta    Exato
1 1.00 50504.95    54541.66 54598.15
2 0.50 52484.90    54583.81 54598.15
3 0.25 53524.12    54594.54 54598.15
4 0.10 54164.26    54597.57 54598.15

 INTERVALO 0 < x   1000
     h        Euler  Runge_Kutta        Exato
1 1.00 1.079790e+20 2.329612e+20 2.353853e+20
2 0.50 1.586147e+20 2.347677e+20 2.353853e+20
3 0.25 1.929724e+20 2.352296e+20 2.353853e+20
4 0.10 2.173342e+20 2.353602e+20 2.353853e+20
```

```r
# Gráficos comparativos
par(mfrow = c(2, 1), mar = c(4, 4, 3, 2))
for (x_final in intervalos) {
  # Usar h = 0.5 para comparação
  euler_data <- euler_method(0.5, x_final)
  rk2_data <- rk2_method(0.5, x_final)
  # Criar pontos para a solução exata
  x_plot <- seq(0, x_final, length.out = min(1000, x_final))
  y_exato_plot <- 1000 * exp(0.04 * x_plot)
  # Determinar limites y adequados
  if (x_final == 100) {
    y_lim <- c(0, 60000)
    titulo <- paste("Intervalo [0,", x_final, "] - h = 0.5")
  } else {
    # Para x = 1000
    y_max <- max(c(euler_data$y, rk2_data$y, y_exato_plot), na.rm = TRUE)
    y_lim <- c(0, min(y_max * 1.1, 1e25))  # Evitando overflow gráfico
    titulo <- paste("Intervalo [0,", x_final, "] - h = 0.5")
  }
  # Plotar solução exato
  plot(x_plot, y_exato_plot, type = "l", lwd = 3, col = "orange", lty = 2,
       xlab = "x", ylab = "y(x)",
       main = titulo,
       ylim = y_lim,
       panel.first = grid(col = "gray80", lty = 3))
  # Adicionar pontos finais para destaque
  lines(euler_data$x, euler_data$y, type = "l", col = "turquoise", lwd = 2)
  lines(rk2_data$x, rk2_data$y, type = "l", col = "purple", lwd = 2)
  points(tail(euler_data$x, 1), tail(euler_data$y, 1),
         pch = 19, col = "turquoise", cex = 1.2)
  points(tail(rk2_data$x, 1), tail(rk2_data$y, 1),
         pch = 19, col = "purple", cex = 1.2)
  points(tail(x_plot, 1), tail(y_exato_plot, 1),
         pch = 19, col = "orange", cex = 1.2)
  legend("topleft",
         legend = c("Solução Exata", "Método de Euler", "Runge-Kutta 2ª Ordem"),
         col = c("orange", "turquoise", "purple"),
         lwd = c(3, 2, 2),
         lty = c(2, 1, 1),
         bg = "white")
}
```






```r
par(mfrow = c(1, 1))
# Análise de erros
cat(" ANÁLISE DE ERROS (h = 0.5)\n")
```

```
 ANÁLISE DE ERROS (h = 0.5)
```

```r
erro_df <- data.frame()
for (x_final in intervalos) {
  euler_sol <- euler_method(0.5, x_final)
  rk2_sol <- rk2_method(0.5, x_final)
  y_exato_points <- 1000 * exp(0.04 * euler_sol$x)
  erro_final_euler <- abs(tail(euler_sol$y, 1) - tail(y_exato_points, 1))
  erro_final_rk2 <- abs(tail(rk2_sol$y, 1) - tail(y_exato_points, 1))
  erro_df <- rbind(erro_df, data.frame(
    Intervalo = x_final,
    Erro_Euler = round(erro_final_euler, 2),
    Erro_Runge_Kutta = round(erro_final_rk2, 2)
  ))
}
print(erro_df)
```

```
  Intervalo   Erro_Euler Erro_Runge_Kutta
```

```
1        100 2.113250e+03     1.434000e+01
2       1000 7.677053e+19     6.175425e+17
```

CONCLUSÃO:

- Em intervalos longos, os erros se acumulam exponencialmente
- Runge-Kutta é mais estável que Euler
- Para x 1000, os erros tornam-se significativos
- Recomenda-se usar h menores em intervalos longos

6 QUESTÃO 5 - EDOs – Problemas de Valor de Contorno (Diferenças Finitas)

6.1 Explicação do conteúdo:

Um PVC é um problema onde queremos resolver uma equação diferencial ordinária (EDO), mas as condições são dadas nas extremidades do intervalo, e não numa única posição.São comuns em calor em barras, vibração, placas metálicas, circuitos elétricos, fluxo de fluidos e vigas em engenharia A ideia é transformar a EDO em um sistema de equações lineares substituindo derivadas por aproximações com diferenças finitas.

Algorítmo:

- *Divisão do intervalo:*

Definimos o passo da malha como $h = \frac{b-a}{n}$.

Com isso, criamos pontos na malha: $x_0 = a$, $x_1 = a+h$, $x_2 = a+2h$, ..., $x_n = b$. Sabemos pelas condições de contorno que $y_0 = y(a) = \alpha$ e também que $y_n = y(b) = \beta$. Queremos encontrar $y_1, y_2, \ldots, y_{n-1}$, que são os valores desconhecidos que aproximam a solução da EDO.

- **Aproximação por diferenças finitas:**

Para a segunda derivada, usamos a fórmula que transforma a EDO em uma equação algébrica:

$$y''(x_i) \approx \frac{y_{i-1} - 2y_i + y_{i+1}}{h^2}$$

- **Montagem da equação num ponto x_i:**

Supondo a EDO $y''(x) = f(x)$ para $i = 1, 2, \ldots, n-1$.

Substituímos no ponto x_i:

$$\frac{y_{i-1} - 2y_i + y_{i+1}}{h^2} = f(x_i)$$

Multiplicando ambos os lados por h^2, chegamos à equação linear do sistema:

$$y_{i-1} - 2y_i + y_{i+1} = h^2 \cdot f(x_i)$$

- *Montagem do sistema linear:*

Fazemos isso para todos os pontos internos → vira um sistema linear tridiagonal:

```r
# Criamos uma matriz 5x5 para representar a estrutura visual com as reticências
matriz_coeficientes <- matrix(c(
  "-2", "1",  "0",  "...", "0",
  "1",  "-2", "1",  "...", ":",
  "0",  "1",  "-2", "...", "0",
  ":",  ".",  ".",  "-2",  "1",
  "0",  "...", "0", "1",   "-2"
), nrow = 5, byrow = TRUE)
# 2. Construção do Vetor Y (Incógnitas)
vetor_y <- matrix(c(
  "Y1",
  "Y2",
  "Y3",
  ":",
  "Yn-1"
), ncol = 1)
# 3. Construção do Vetor B (Lado Direito da equação)
# Representa os termos h^2*f(x) e as condições de contorno y_0 e y_n
vetor_b <- matrix(c(
  "h^2 f(X1) - Y0",
  "h^2 f(X2)",
  "h^2 f(X3)",
  ":",
  "h^2 f(Xn-2)",
  "h^2 f(Xn-1) - Yn"
), ncol = 1)
# Exibindo as matrizes sem aspas para visualização limpa
cat(" Matriz de Coeficientes \n")
```

```
 Matriz de Coeficientes
```

```r
print(noquote(matriz_coeficientes))
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,] -2   1    0    ...  0
[2,] 1    -2   1    ...  :
[3,] 0    1    -2   ...  0
[4,] :    .    .    -2   1
[5,] 0    ...  0    1    -2
```

```
cat("\n Vetor Y \n")
```

```
 Vetor Y
```

```
print(noquote(vetor_y))
```

```
     [,1]
[1,] Y1
[2,] Y2
[3,] Y3
[4,] :
[5,] Yn-1
```

```
cat("\n Vetor Resultado (B) \n")
```

```
 Vetor Resultado (B)
```

```
print(noquote(vetor_b))
```

```
     [,1]
[1,] h^2 f(X1) - Y0
[2,] h^2 f(X2)
[3,] h^2 f(X3)
[4,] :
[5,] h^2 f(Xn-2)
[6,] h^2 f(Xn-1) - Yn
```

Os termos $Y(0) = \alpha$ e $Yn = \beta$ entram no lado direito

Depois é só resolver o sistema e obter todas as aproximações Y1, … , Yn-1

Resumindo:

- Defina o intervalo [a,b]
- Escolha o número de divisões n
- Calcule h
- Crie os pontos Xi
- Substitua as derivadas por diferenças finitas

- Monte o sistema linear
- Aplique métodos de álgebra linear (solve, Gauss, Thomas, etc.)
- Obtenha os valores aproximados de y

Diferenças finitas transformam derivadas em equações algébricas, permitindo que:

- EDO + condições de contorno → sistema linear → solução numérica
- Quanto mais pontos (maior n), menor o erro

6.2 *Resolução da questão:*

6.3 Questão 5a:

```
# 1. Definição dos Parâmetros
h <- 0.01
x_min <- 0
x_max <- 1
N <- (x_max - x_min) / h  # Número de subintervalos (100)
pontos <- N + 1           # Total de pontos (101: x0 a x100)
# Vetor de x (todos os pontos)
x <- seq(x_min, x_max, by = h)
# 2. Solução Exata (dada no enunciado)
y_exata_func <- function(x) {
  return(2 * exp(-x) * (1 - x) + x - 2)
}
y_exata <- y_exata_func(x)
# 3. Construção do Sistema Linear (MDF)
# Precisamos resolver para os pontos INTERNOS: x_1 até x_{N-1}
# Dimensão do sistema linear
n_incognitas <- N - 1
# Coeficientes da equação de diferenças:
# (1 - h)y_{i-1} + (h^2 - 2)y_i + (1 + h)y_{i+1} = h^2 * x_i
coef_inf <- 1 - h      # Coeficiente de y_{i-1}
coef_diag <- h^2 - 2   # Coeficiente de y_{i}
coef_sup <- 1 + h      # Coeficiente de y_{i+1}
# Inicializa Matriz A (Preenchida com zeros)
A <- matrix(0, nrow = n_incognitas, ncol = n_incognitas)
# Inicializa Vetor b
b <- numeric(n_incognitas)
# Preenchimento da Matriz A e Vetor b
# Os índices da matriz 'i' vão de 1 a 99, que correspondem aos x_1 a x_99
```

```r
for (i in 1:n_incognitas) {
  # O x correspondente a esta linha da matriz é x[i+1] pois x[1] é o x_0
  x_atual <- x[i+1]
  # Diagonal Principal
  A[i, i] <- coef_diag
  # Diagonal Inferior (se não for a primeira linha)
  if (i > 1) {
    A[i, i-1] <- coef_inf
  }
  # Diagonal Superior (se não for a última linha)
  if (i < n_incognitas) {
    A[i, i+1] <- coef_sup
  }
  # Lado direito da equação (h^2 * x_i)
  b[i] <- h^2 * x_atual
}
# 4. Aplicação das Condições de Contorno (Fronteiras)
y_0 <- 0   # y(0) = 0
y_N <- -1  # y(1) = -1
# Ajuste na primeira equação (i=1) -> termo y_0 passa para direita
b[1] <- b[1] - (coef_inf * y_0)
# Ajuste na última equação (i=99) -> termo y_N passa para direita
b[n_incognitas] <- b[n_incognitas] - (coef_sup * y_N)
# 5. Resolução do Sistema Linear
# Método escolhido: Solvedor padrão do R (baseado em decomposição LU da LAPACK)
y_interno <- solve(A, b)
# Montagem da solução final (incluindo as bordas conhecidas)
y_numerica <- c(y_0, y_interno, y_N)
# 6. Cálculo do Erro Absoluto
erro_absoluto <- abs(y_numerica - y_exata)
# 7. Apresentação dos Resultados
# Dataframe para visualização
resultados <- data.frame(
  x = x,
  y_Numerico = y_numerica,
  y_Exata = y_exata,
  Erro_Abs = erro_absoluto
)
# Mostrar as primeiras e últimas linhas e o erro máximo
print(head(resultados))
```

```
     x  y_Numerico     y_Exata     Erro_Abs
```

```
1 0.00  0.00000000  0.00000000 0.000000e+00
2 0.01 -0.02970215 -0.02970133 8.160293e-07
3 0.02 -0.05881220 -0.05881060 1.597897e-06
4 0.03 -0.08733801 -0.08733566 2.346428e-06
5 0.04 -0.11528734 -0.11528428 3.062432e-06
6 0.05 -0.14266784 -0.14266409 3.746706e-06
```

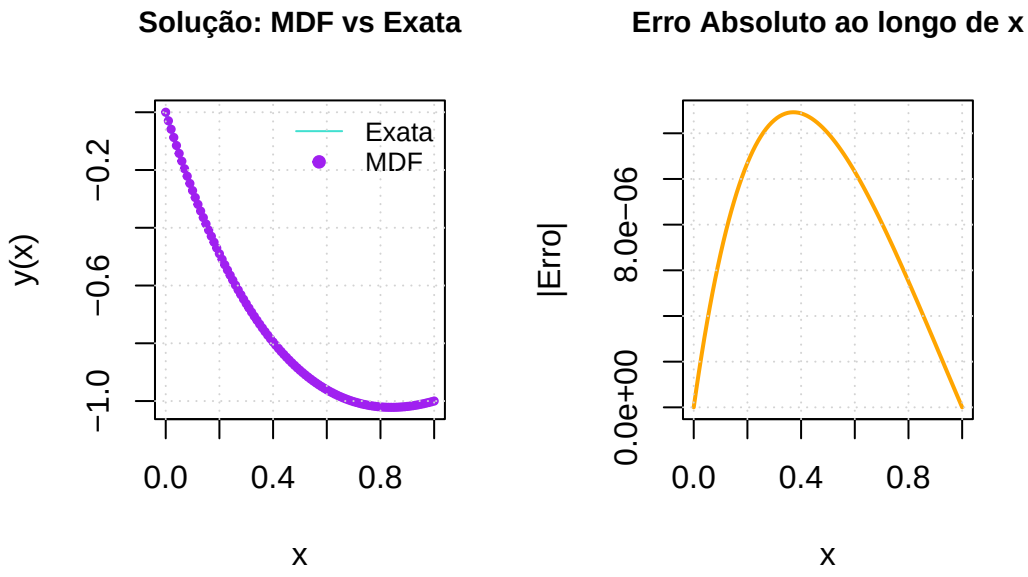
```r
print(tail(resultados))
```

```
       x y_Numerico   y_Exata     Erro_Abs
96  0.95  -1.011327 -1.011326 1.385484e-06
97  0.96  -1.009370 -1.009369 1.107684e-06
98  0.97  -1.007256 -1.007255 8.301449e-07
99  0.98  -1.004988 -1.004988 5.529588e-07
100 0.99  -1.002569 -1.002568 2.762149e-07
101 1.00  -1.000000 -1.000000 0.000000e+00
```

```r
cat("Erro Máximo Absoluto:", max(erro_absoluto), "\n")
```

```
Erro Máximo Absoluto: 1.292183e-05
```

```r
# Gráfico Comparativo
# Ajusta a área de desenho para mostrar 2 gráficos lado a lado
# (Isso facilita para você tirar o print para o relatório)
par(mfrow = c(1, 2))
# --- Gráfico 1: Comparação das Curvas ---
plot(x, y_exata, type = "l", col = "turquoise", lwd = 2,
     main = "Solução: MDF vs Exata",
     xlab = "x", ylab = "y(x)",
     cex.main = 0.9) # Tamanho do título
points(x, y_numerica, col = "purple", pch = 19, cex = 0.5)
legend("topright", legend = c("Exata", "MDF"),
       col = c("turquoise", "purple"), lty = c(1, NA), pch = c(NA, 19),
       bty = "n", cex = 0.8)
grid()
# --- Gráfico 2: Erro Absoluto ---
plot(x, erro_absoluto, type = "l", col = "orange", lwd = 2,
     main = "Erro Absoluto ao longo de x",
     xlab = "x", ylab = "|Erro|",
     cex.main = 0.9)
grid()
```




```
# Restaura a configuração original (1 gráfico por vez) caso precise depois
par(mfrow = c(1, 1))
```

Qual foi o método escolhido para a resolução do sistema linear? Justifique

- O método da Decomposição LU é numericamente estável para matrizes não singulares bem condicionadas. Ao usar a função nativa **solve()**, aproveitamos implementações altamente otimizadas e testadas da biblioteca LAPACK, evitando erros de implementação manual de algoritmos.
- O sistema linear gerado tem dimensão 99X99. Embora métodos diretos gerais (como LU ou Eliminação de Gauss) tenham complexidade O(N^3) para um N=99 o tempo de execução é desprezível (frações de segundo) em computadores modernos. Portanto, não houve gargalo computacional que exigisse um método iterativo.
- A matriz do sistema A é tridiagonal (possui zeros em todas as posições exceto na diagonal principal e nas duas subdiagonais adjacentes).

CONCLUINDO: Para a resolução do sistema linear AY = b, utilizou-se o Método direto de Decomposição LU (via função solve do R). A escolha justifica-se pela robustez do algoritmo (baseado na biblioteca LAPACK) e pela dimensão reduzida do sistema (99 X 99), onde o custo computacional O(N^3) é irrelevante, dispensando a necessidade de implementações específicas para matrizes esparsas como o Algoritmo de Thomas.

6.4 Questão 5b:

Por que os maiores valores de erros obtidos são os correspondentes aos valores centrais de x?

O erro é maior no centro porque o problema é um Problema de Valor de Contorno (PVC) com condições de Dirichlet fixas nas extremidades.

- *Fixação nas Bordas (Condições de Contorno):* No método das Diferenças Finitas, impusemos os valores exatos da solução nas fronteiras: Y(0)=0 e Y(1) = -1 Isso significa que, por definição, o erro numérico em X=0 e X=1 é nulo (como visto na tabela e no gráfico).
- *Acúmulo de Erro de Truncamento:* O método aproxima as derivadas em cada ponto interno usando os vizinhos. Cada aproximação carrega um pequeno erro de truncamento (da ordem de O(h^2)).Nos pontos próximos às bordas (ex: X=0.01 ou X=0.99), o cálculo depende diretamente de um valor "perfeito" (a borda) e um valor aproximado. Isso "ancora" a solução numérica, mantendo o erro baixo.À medida que nos afastamos das bordas e caminhamos para o centro do intervalo, o cálculo em um ponto Xi depende de vizinhos Xi-1 e Xi+1 que também são aproximados.
- Os erros se propagam e se acumulam a partir das bordas em direção ao interior. O centro do intervalo (X é aproximadamente 0.5) é a região "mais distante" das informações exatas (as fronteiras). Portanto, é onde a solução numérica tem maior liberdade para se desviar da solução exata, resultando no pico de erro observado.

CONCLUINDO:

Observa-se que os erros absolutos são nulos nas extremidades X=0 e X=1 e crescem em direção ao centro do intervalo, atingindo o máximo próximo a X=0.5. Isso ocorre devido à natureza do Problema de Valor de Contorno com condições de Dirichlet: os valores nas fronteiras são fixados exatamente. O erro de truncamento local, inerente às aproximações de diferenças finitas, se propaga pelo domínio. Como os pontos centrais são os mais distantes das fronteiras (onde a informação é exata), eles sofrem o maior acúmulo de imprecisões numéricas, resultando no perfil de erro parabólico observado.

6.5 Questão 5c:

Verifique numericamente que o Método das Diferenças Finitas para resolver Problemas de Valor de Contorno tem ordem 2.

Para verificar numericamente que o método tem ordem 2, precisamos demonstrar que o erro global é proporcional ao quadrado do passo h.Em termos práticos: Se dividirmos o passo h pela metade, o erro deve diminuir aproximadamente 4 vezes (já que (2^2)=4).

```r
# 1. Encapsular o método numa função para facilitar o teste com vários h
resolver_pvc_erro <- function(h) {
  x_min <- 0; x_max <- 1
  N <- (x_max - x_min) / h
  n_incognitas <- N - 1
  x <- seq(x_min, x_max, by = h)

  # Coeficientes
  coef_inf <- 1 - h
  coef_diag <- h^2 - 2
  coef_sup <- 1 + h

  # Montar Matriz A e vetor b
  A <- matrix(0, nrow = n_incognitas, ncol = n_incognitas)
  b <- numeric(n_incognitas)

  for (i in 1:n_incognitas) {
    x_atual <- x[i+1]
    A[i, i] <- coef_diag
    if (i > 1) A[i, i-1] <- coef_inf
    if (i < n_incognitas) A[i, i+1] <- coef_sup
    b[i] <- h^2 * x_atual
  }

  # Condições de Contorno
  y_0 <- 0; y_N <- -1
  b[1] <- b[1] - (coef_inf * y_0)
  b[n_incognitas] <- b[n_incognitas] - (coef_sup * y_N)

  # Resolver
  y_interno <- solve(A, b)
  y_numerica <- c(y_0, y_interno, y_N)

  # Solução Exata
  y_exata <- 2 * exp(-x) * (1 - x) + x - 2

  # Retorna o Erro Máximo
  return(max(abs(y_numerica - y_exata)))
}

# 2. Testar para uma sequência de passos (h) decrescentes
# Vamos dividir h por 2 sucessivamente
```

```r
h_valores <- c(0.1, 0.05, 0.025, 0.0125, 0.00625)
erros <- numeric(length(h_valores))

for (k in 1:length(h_valores)) {
  erros[k] <- resolver_pvc_erro(h_valores[k])
}

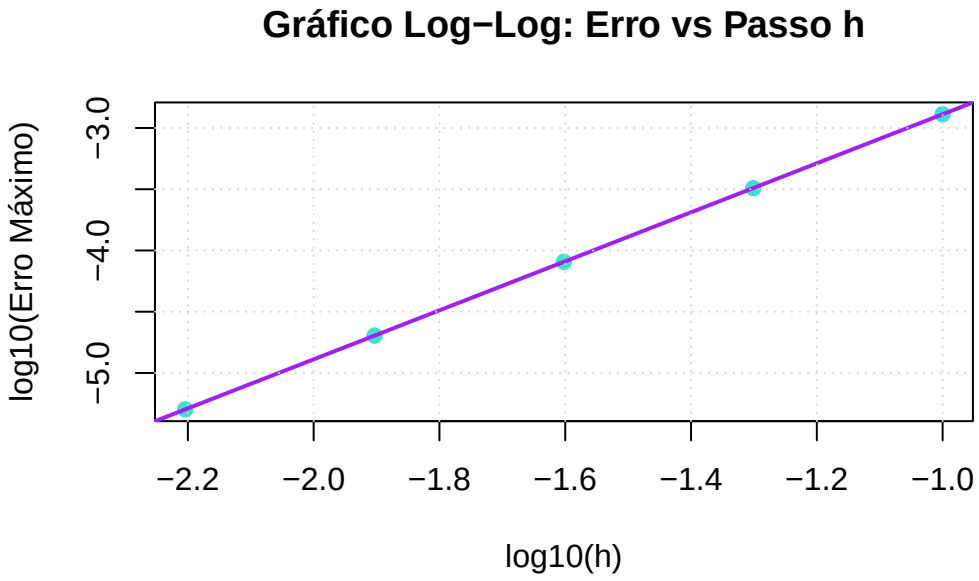
# 3. Calcular a redução do erro (Razão) e a Ordem Estimada
razao <- c(NA, erros[1:(length(erros)-1)] / erros[2:length(erros)])
ordem_estimada <- log2(razao) # log na base 2 da razão dá a ordem p

# 4. Criar Tabela de Resultados
tabela_convergencia <- data.frame(
  h = h_valores,
  Erro_Max = erros,
  Razao_Erro = razao,
  Ordem_Aproximada = ordem_estimada
)

print(tabela_convergencia)
```

```
        h     Erro_Max Razao_Erro Ordem_Aproximada
1 0.10000 1.294593e-03         NA               NA
2 0.05000 3.227841e-04   4.010709         2.003857
3 0.02500 8.078127e-05   3.995779         1.998477
4 0.01250 2.018973e-05   4.001107         2.000399
5 0.00625 5.047297e-06   4.000107         2.000039
```

```r
# 5. Gráfico Log-Log (Prova Visual da Ordem)
# Se a ordem for 2, a inclinação da reta deve ser 2
plot(log10(h_valores), log10(erros), pch = 19, col = "turquoise",
     main = "Gráfico Log-Log: Erro vs Passo h",
     xlab = "log10(h)", ylab = "log10(Erro Máximo)")
abline(lm(log10(erros) ~ log10(h_valores)), col = "purple", lwd = 2)
grid()
```




```r
# Cálculo da inclinação exata da reta (slope)
modelo <- lm(log10(erros) ~ log10(h_valores))
cat("\nInclinação da reta (Ordem de Convergência):", coef(modelo)[2], "\n")
```

```
Inclinação da reta (Ordem de Convergência): 2.000442
```

- Para verificar a ordem do método, calculou-se o erro máximo absoluto para uma sequência de passos h sucessivamente divididos pela metade (h, h/2, h/4,...).Teoricamente, para um método de ordem p, espera-se que o erro E(h) comporte-se como $E(h) \approx Ch^p$.. Ao comparar o erro de um passo h com o erro do passo h/2, temos a razão:

$$\frac{E(h)}{E(h/2)} \approx \frac{C \cdot h^p}{C \cdot (h/2)^p} = 2^p$$

- Os resultados numéricos obtidos mostram que a razão entre os erros consecutivos é aproximadamente 4. Calculando-se $\log_2(4)$, confirma-se que $p \approx 2$. Além disso, o gráfico log-log de Erro versus h apresentou uma reta com inclinação igual a 2, validando numericamente que o Método das Diferenças Finitas centrado utilizado possui convergência quadrática ($O(h^2)$).

Também podemos tirar essa conclusão baseando-nos nos seguintes pontos teóricos:

- Da fórmula da primeira derivada centrada:

$$y'(x_i) \approx \frac{y_{i+1} - y_{i-1}}{2h}$$

  Sendo esta uma aproximação $O(h^2)$, supondo que $y'''(x)$ seja limitada em $[a, b]$, conclui-se que o erro é de ordem h^2.

- Da expressão da Série de Taylor para a segunda derivada:

$$y''(x_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}$$

  Que possui erro da ordem h^2, desde que a quarta derivada $y^{(4)}(x)$ seja limitada em $[a, b]$.

- Encontrando a parábola $p_2(x)$ que interpola $y(x)$ nos pontos vizinhos x_{i-1}, x_i e x_{i+1}. Prova-se geometricamente que o erro de aproximação local é de ordem quadrática.

6.6 Questão 5d:

```r
# Análise de Viabilidade Computacional
analisar_viabilidade <- function(h) {
  # Número de incógnitas (N-1)
  N <- 1/h
  n_incognitas <- N - 1
  # Tamanho de um número double (8 bytes)
  bytes_por_num <- 8
  # 1. Memória para Matriz Densa (Padrão do R: solve())
  total_elementos_densa <- n_incognitas^2
  memoria_densa_gb <- (total_elementos_densa * bytes_por_num) / (1024^3)
  # 2. Memória para Matriz Esparsa (Apenas 3 diagonais)
  # Armazena apenas 3 vetores de tamanho N
  total_elementos_esparsa <- 3 * n_incognitas
  memoria_esparsa_mb <- (total_elementos_esparsa * bytes_por_num) / (1024^2)
  # 3. Estimativa de Operações (Complexidade)
  # Solver Denso (Decomposição LU): ~ (2/3) * N^3
  ops_densa <- (2/3) * n_incognitas^3
  # Solver Esparso (Algoritmo de Thomas): ~ 8 * N
  ops_esparsa <- 8 * n_incognitas
  if(memoria_densa_gb > 16) {
    cat("CONCLUSÃO: IMPOSSÍVEL rodar com 'solve()' padrão (Estouro de Memória).\n")
  } else {
    cat("CONCLUSÃO: Viável rodar.\n")
  }
```

```
}
# Testar os valores pedidos
analisar_viabilidade(0.00001)       # h = 10^-5
```

```
CONCLUSÃO: IMPOSSÍVEL rodar com 'solve()' padrão (Estouro de Memória).
```

```
analisar_viabilidade(0.000000001)   # h = 10^-9
```

```
CONCLUSÃO: IMPOSSÍVEL rodar com 'solve()' padrão (Estouro de Memória).
```

CONCLUINDO:

- Ao tentar executar o algoritmo padrão para h=10^{-5}, o sistema retorna um erro de alocação de memória (ou trava), pois a matriz de coeficientes A densa exigiria aproximadamente 74.5 GB de RAM, superando a capacidade do hardware utilizado.
- Para h=10^{-9}, a inviabilidade é absoluta, tanto por memória quanto por tempo de processamento.

Discussão sobre Memória:

- O método implementado utiliza armazenamento denso (N X N). O consumo de memória cresce quadraticamente (O(N^2)).
- Para h=0.01 (N=100), a matriz ocupa aproximadamente 80 KB (insignificante).
- Para h=10^{-5} (N=10^5), a matriz ocuparia aproximadamente 74.5 GB.
- A solução seria utilizar estruturas de Matrizes Esparsas (armazenando apenas as diagonais não nulas), onde o consumo de memória cresce linearmente (O(N)), reduzindo a necessidade para meros 2.3 MB no caso de h=10^{-5}.

Discussão sobre Ordem de Operações e Tempo:

- A resolução direta de um sistema linear cheio (Decomposição LU) possui complexidade de tempo cúbica O(N^3).
- Passar de N=100 para N=100.000 (aumento de 1000x no tamanho) aumentaria o tempo de processamento em 1000^3 = 1 bilhão de vezes, tornando o método denso impraticável.
- Utilizando algoritmos específicos para matrizes tridiagonais (como o Algoritmo de Thomas), a complexidade seria reduzida para O(N), tornando a resolução para h=10^{-5} instantânea.

Assim, para malhas muito finas (h muito pequeno), é obrigatório o uso de métodos que explorem a esparsidade da matriz (como o Algoritmo de Thomas ou métodos iterativos como Gauss-Seidel), sob pena de estouro de memória e tempo de execução infinito."

7 CONCLUSÃO:

O presente trabalho computacional permitiu consolidar a compreensão teórica e prática de métodos numéricos fundamentais, utilizando a linguagem R como ferramenta de implementação e análise gráfica. Ao longo das atividades, foi possível observar como diferentes técnicas abordam a aproximação de funções e a resolução de equações diferenciais, destacando-se os seguintes pontos:

- Interpolação e Ajuste de Curvas: A comparação entre a Interpolação de Lagrange e Splines Lineares evidenciou o compromisso entre a suavidade da curva e a estabilidade numérica.
- No ajuste de curvas pelo Método dos Quadrados Mínimos, confirmou-se que a escolha adequada do modelo (como o quadrático frente ao linear) é crucial para a minimização dos erros e melhor representação do fenômeno físico.
- Integração Numérica: A análise crítica entre as regras dos Trapézios e de Simpson demonstrou a superioridade do método de Simpson para funções suaves. Verificou-se que, devido à sua ordem de erro superior (O(h^4) contra O(h^2)), a regra de Simpson atinge maior precisão com menor custo computacional, sendo a escolha preferencial quando a função permite.
- Solução de EDOs (PVI): No estudo de Problemas de Valor Inicial, ficou claro que métodos de ordem mais alta, como Runge-Kutta (RK2 e RK4), oferecem um equilíbrio muito mais vantajoso entre precisão e eficiência do que o Método de Euler. Enquanto Euler exige passos extremamente pequenos para manter a precisão (o que pode ser inviável computacionalmente), os métodos de Runge-Kutta conseguem resultados robustos com passos maiores.
- Solução de EDOs (PVC) e Diferenças Finitas: A resolução do Problema de Valor de Contorno ilustrou como equações diferenciais podem ser transformadas em sistemas lineares. A análise de erro confirmou a convergência quadrática (O(h^2)) do Método das Diferenças Finitas e explicou o comportamento do erro máximo no centro do intervalo, longe das condições de contorno fixas. Além disso, a discussão sobre complexidade computacional ressaltou a importância vital de utilizar algoritmos para matrizes esparsas (como o Algoritmo de Thomas) em malhas finas, evitando o estouro de memória e processamento inviável associado a métodos densos (O(N^3)).
- Em suma, o trabalho evidenciou que a escolha do método numérico não deve ser arbitrária, mas sim baseada na análise do problema, na precisão requerida e nos recursos computacionais disponíveis. A implementação dos algoritmos reforçou a capacidade de transformar formulações matemáticas abstratas em soluções práticas e quantificáveis.

8 BIBLIOGRAFIA:

- Slides de Aula
- Mácia A. Gomes Ruggiero; vera Lúcia da Rocha Lopes – *Cálculo Numérico - Aspectos Teóricos e Computacionais (2ºa edição)*
- Burden & Faires – *Análise Numérica*
- Documentação R e pacotes utilizados